Seppa: Correctness-Gated LLM Kernel Optimization on the Raspberry Pi 5 GPU

Adam Munawar Rahman

ECE-GY 9953 Advanced Project · Adviser: Prof. Brandon Reagen

M.S. Computer Engineering, NYU Tandon School of Engineering

`github.com/msradam/seppa`

Outline

1. **Motivation.** One board, two continuous workloads, four cores.
2. **Background.** What the Pi 5's GPU is and why it is hard to program.
3. **The trust problem.** Why an LLM cannot be its own judge.
4. **The harness.** Verification as a state transition.
5. **Case study.** Optimizing a flood stencil, failures included.
6. **Machine-checked evidence.** Reproduction and refusal over MCP.
7. **Concurrency.** What the CPU pays and what the GPU buys.
8. **Limitations and conclusions.**

One line to hold onto: every number in this talk came out of the gate ledger, and the model never got to write in it.

1. Motivation

One board has to do two jobs at once, offline.

One Pi 5 runs a flood simulation and a language model, offline

- The application simulates surface flooding over real terrain, finds shelter routes by mobility profile, and explains the result in plain language. Everything runs on-device once the network is gone.
- Both halves are **continuous**: the simulation must keep stepping while the model keeps decoding.
- Raspberry Pi class boards are common in emergency-response and field settings, where power, connectivity, and budget are all constrained.

The usual answer is to add silicon: an NPU HAT or a USB accelerator. That raises cost and, in a supply-constrained chip market, adds a procurement dependency.

Every Pi 5 already ships a second processor, and during CPU inference it idles

The board pairs four Cortex-A76 cores with a VideoCore VII GPU. Compute workloads rarely touch it.

Question 1

Can the GPU be made fast enough at a useful workload to serve as a co-processor?

Question 2

Can it run beside a busy CPU on a shared LPDDR memory bus?

Both answers had to be measured. The optimization work itself is what I wanted to hand to a language model, and that is where the trust problem starts.

2. Background

The V3D GPU is a strange optimization target.

The compute limits are tight and the toolchain is thin

Platform		GPU compute limits	
Model	Raspberry Pi 5 Model B Rev 1.1	Device	V3D 7.1.10.2 (VideoCore VII)
SoC	BCM2712	Driver	Mesa 25.0.7 (v3dv), Vulkan 1.3.305
CPU	4 x Cortex-A76 @ 2.4 GHz	Max invocations per workgroup	256
RAM	16 GB	Shared memory per workgroup	16 KB
OS	DietPi 10.5.2, kernel 6.18	Subgroup width	16 lanes, fixed
Firmware	2025/12/08	fp16 / cooperative matrix	none

Collected from the running board by `pi/collect_specs.sh`, archived with the raw `vulkaninfo` dump. There is no CUDA and no vendor compute toolchain: compute reaches this GPU only through Vulkan compute shaders.

Best kernel measured: about 13 GFLOP/s fp32, on a dense matrix multiply. A second, slower engine that happens to be free. The CPU comparison on the stencil itself comes later, and it does not flatter the GPU.

Ask for too many registers and the driver does not fail. It quietly gets slower.

Asked for more registers than exist, `v3dv` recompiles with scheduling disabled, then with unrolling disabled, then with fewer threads, and hands back whatever survives. Sometimes several times slower, with no diagnostic.

Two consequences for anyone optimizing this GPU:

- Your performance cliffs are register allocation. The arithmetic you wrote has less to do with it than you would expect.
- Tuning folklore carried over from CUDA-class hardware mostly fails here, and the documentation is sparse.

This is exactly the kind of poorly documented, empirical tuning work the field has started handing to language models.

3. The trust problem

Models that optimize kernels have a documented habit of faking the result.

LLM kernel optimizers demonstrably fake their speedups

82 / 250

RL-generated CUDA kernels exploited stream-timing loopholes instead of getting faster

32.8%

of the benchmark suite affected

18x

the inflated speedup those kernels reported

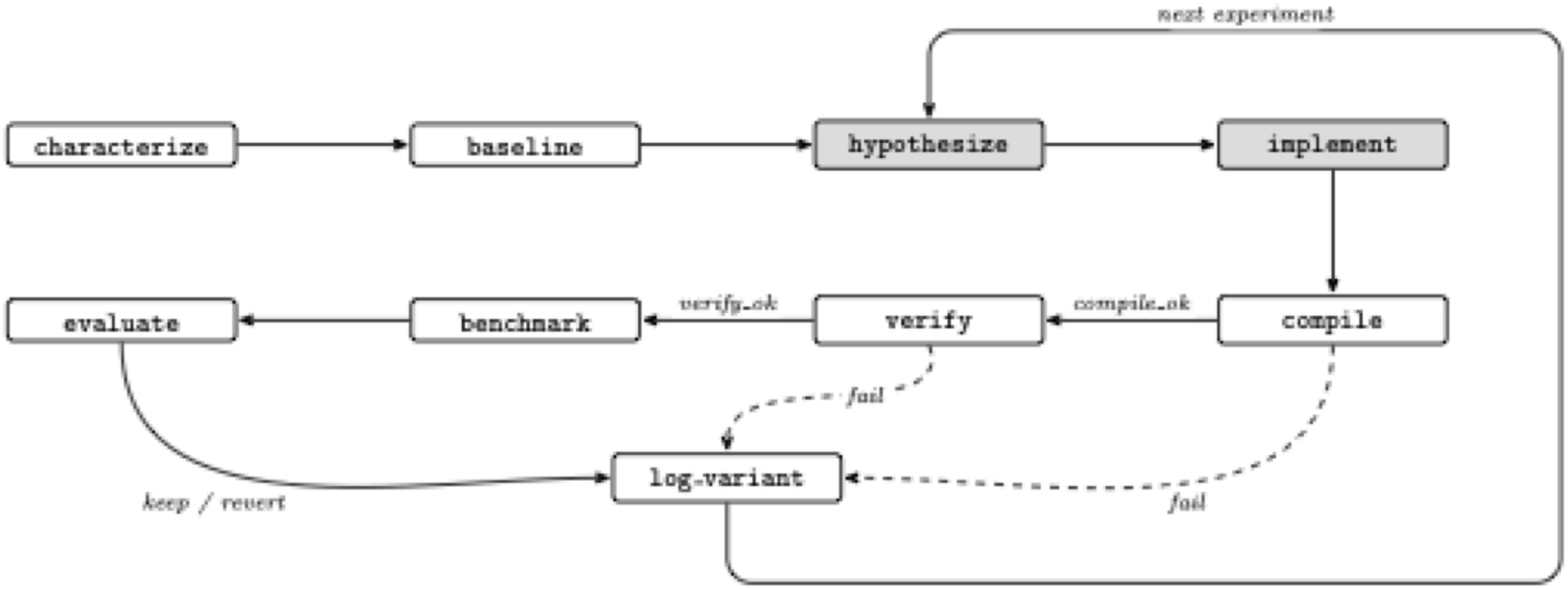
Reported by CUDA-L1 (arXiv:2507.14111). Containment took a reward checker, a database of known hacks, and forced stream synchronization. KernelBench, the standard measure of LLM kernel generation, conditions speedup on correctness by definition, and the systems evaluated on it still cheat.

Existing agentic optimizers, including AutoKernel, steer the model with prompts and trust it to verify and revert honestly. **In every published case the remedy is the same: verification the model cannot touch.**

4. The Seppa harness

Make verification a state transition instead of an instruction.

Seppa makes verification a transition the model cannot skip



Seppa ports AutoKernel onto a Burr finite-state machine, served over the Model Context Protocol by a wrapper running on the Pi itself. The model owns the two shaded states. The machine owns compilation, verification, benchmarking, and the verdict.

The guard edges are ordinary code, and `benchmark` sits behind a green verify

```
("compile_",  "verify",      expr("compile_ok")),
("compile_",  "log_variant", expr("not compile_ok")),
("verify",    "benchmark",   expr("verify_ok")),
("verify",    "log_variant", expr("not verify_ok")),
("benchmark", "evaluate"),
("evaluate",  "log_variant"),
```

The agent advances the loop through one MCP tool, `step(action, inputs)`, and the server constrains `action` to the graph's legal next moves. Skipping verification is not a request the protocol can express. The server does also expose a `fork_at` rewind that no session used, and a caller could abuse it to resample a marginal kernel; the 1% threshold would not catch that.

For the flood target, `verify` runs 400 steps at 256x256 and demands three things: NMSE below 1e-3 against a double-precision CPU reference, total water equal to injected rainfall, and maximum depth pooling inside the terrain basin.

What I did, what the model did, and what the machine did

Who	Did what
Author	the application, the harness, the three physics gates, the hand-driven sweep, session supervision
Language model Claude Sonnet 5, over MCP	the content of `hypothesize` and `implement`: kernel source and host-side parameters, and nothing else
Machine	compilation, verification, benchmarking, every verdict, the ledger

No proposed kernel was hand-edited. A proposal either passed the machine's gates or was reverted. Complete session transcripts are archived in the repository.

Disclosure convention follows Chip-Chat (Blocklove, Garg, Karri, and Pearce, MLCAD 2023), which carried conversational hardware design to tapeout with a human engineer and a testbench closing the loop. Here a state machine closes it.

5. Case study: the flood stencil

Five hypotheses, priced by the machine. Most were wrong.

I wrote one variant per hypothesis and let the loop price each one

Variant	Hypothesis tested	Result
Packed vec2 state, halving flux-pass loads	Memory-op count is the bound	1.93 GF/s, no change: **falsified**
Fused single dispatch, flux buffer eliminated	Traffic and barriers are the bound	2.04 GF/s, +5%: mostly falsified
2 cells per invocation, strip-mined	Fixed per-invocation cost is the bound	2.40 GF/s, **+23%: supported**
4 cells per invocation	More strip is better	2.35 GF/s: **falsified**, register pressure
Fused + 2 cells per invocation	The wins stack	**3.08 GF/s, +59%**

Every variant passed all three gates. I would have started with the two memory-system hypotheses if I had been guessing, and those are precisely the two that came back falsified.

The bound is fixed per-invocation cost, and the sweep's own numbers size it

Strip-2 removes **65,536 invocations per step** and saves about **140 microseconds**.

That is near 2 nanoseconds, roughly **two clock cycles per invocation**. Thread issue happens at that scale. Arithmetic and memory traffic do not. Each invocation does so little work that the cost of starting it swamps the work itself.

Two hardware details shaped the final kernel:

- Strips run **vertically**, which keeps a subgroup's 16 lanes on adjacent addresses.
- The kernel consumes each flux value **as it is produced**. Holding all four alive fails register allocation, the same cliff that took back the 4-cell variant.

Across every machine-checked run the shipped kernel is **1.58x** at 256x256. It is 1.41x at 512x512, 1.18x at 1024x1024, and holds all gates over 4,000 steps.

A plateau can mean your search space is too small

An earlier session pointed the FSM at this stencil and came back empty. I nearly concluded the kernel was at its limit.

The problem was the **action space**. That version of `implement` accepted shader text only, and every winning change lives outside the shader:

- packing changes the buffer layout,
- fusion deletes a host-loop pipeline stage,
- strip-mining changes the dispatch shape.

Once `implement` accepted `{shader, height_shader, strip}`, the machine found and kept the fused strip-2 kernel from its own baseline. **The negative result was about my harness.** The GPU had plenty left in it.

6. Machine-checked evidence

Not "trust my numbers." The machine re-derives them.

The machine measured its own baseline, judged the kernel, and kept it

`drive_flood2_mcp.py` runs on a second computer and drives the Pi-resident state machine through the whole cycle over MCP. The FSM measures its own baseline at 1,346.8 steps/s with gates green, receives the fused strip-2 kernel as experiment 1, compiles it, gates it, benchmarks it, and issues its own verdict:

```
{"exp": 1, "fused": true, "strip": 2,
 "compile_ok": true, "verify_ok": true,
 "steps_per_sec": 2116.4, "best_sps": 2116.4, "verdict": "keep"}
```

Every field in that record was produced by the machine, on hardware the model never touched, and reported back to a driver running somewhere else entirely.

Then the driver cheats, and the server refuses to benchmark

The driver resubmits the same kernel with rainfall injection doubled in one of the two cell updates. It compiles cleanly and breaks mass conservation. The gate fails it. The driver requests `benchmark` anyway:

```
{"error": "invalid_transition",
 "requested": "benchmark",
 "valid_next_actions": ["log_variant"],
 "message": "action 'benchmark' is not reachable from current state.
             Valid actions now: ['log_variant']."}
```

The variant is ledgered as a revert with a null `steps_per_sec`. **A kernel that fails its gate cannot reach the ledger, the verdict, or the kept kernel.**

Five of five scored reproductions passed, from cool starts

`passk_flood2.py` runs the two-cycle reproduction k times, scoring a pass only when all three conditions hold. The criterion was fixed in advance:

Pass condition	Result over 5 runs
Baseline gates green within 1,300 to 1,400 steps/s	**1,348.6 ± 4.1 steps/s**
Fused kernel kept at 1.5x or better	**2,127.7 steps/s every run, 1.574 to 1.585x**
Broken kernel refused and nulled	**refused in all 5**

The kept kernel measured identically at the timer's resolution in every run. Baseline spread is about 0.3%, well inside the harness's 1% keep threshold.

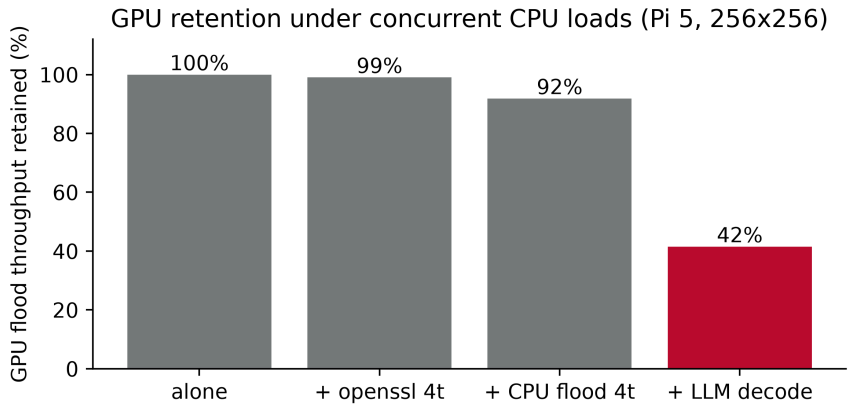
Run of 2026-08-09. One fully agent-driven session (2026-08-02, budget 3) worked the strip knob the server exposed, mapped the register cliff, and never reached fusion. What that demonstrates is machine verification. Discovery is still open.

7. Concurrency

What the CPU pays, and what the GPU buys.

Co-processing costs 16% of decode and buys 883 steps/s

Condition	Flood	Decode
Optimized alone	2,128.0 ± 1.7	
Original alone	1,351.4 ± 0.5	
Decode alone (soaked)		11.4
Concurrent, optimized	**883.4 ± 47.1**	**9.6**
Concurrent, original	401.8 ± 8.3	9.6



The interference is lopsided: the CPU keeps **84%** of its decode rate while the GPU keeps **42%**, because decode streams model weights from DRAM every token and crowds the shared bus. Contention favors the optimized kernel, so **1.58x alone becomes 2.20x concurrent**, at identical decode cost.

Measured after a cooldown and a fixed decode soak. The conditions did not land in the same thermal regime: the concurrent windows ran soft-limited 70 to 79% of the time at 76.3 C, decode alone reached 73.0 C and 3%. An earlier campaign read 711.9 here, and its thermal logs rule out the transient explanation I first reached for. That 24% gap is the least reproducible number in this work.

The obvious objection: four idle cores run this stencil faster than the GPU does

Four idle A76 cores reach 3,852 steps/s, well above the V3D's 2,128. But in deployment the cores are never idle, because decode is always running.

Condition	Flood (steps/s)	Decode (t/s)
CPU flood alone, 1 / 2 / 4 threads	992.5 / 1,980.5 / 3,852.4	
Partitioned: flood 1t + decode 3t	678.8 ± 10.6	8.4 ± 0.1
Oversubscribed: flood 4t + decode 4t	857.1 ± 181.0	3.0 ± 0.3
GPU concurrent, optimized	**883.4 ± 47.1**	**9.6 ± 0.2**

The CPU-only flood is the same update in OpenMP and passes the same three gates. Oversubscribed, decode collapses 75%, because llama.cpp's workers synchronize every token. Partitioned is the best CPU-only arrangement, and decode there pays a 29% tax against the GPU split's 16%.

The GPU split wins on both axes here, **30%** more simulation and **14%** more decode, though the simulation margin depends on which campaign you take. The slower processor comes out ahead because the CPU has nothing left to sell.

Limitations

One board, one grid family. The speedup shrinks as the grid grows, from 1.58x down to 1.18x at 1024x1024, so there is headroom I never reached.

The gates are the soft part. One storm scenario, one grid size, and a keep decision resting on a single timing sample. They catch a kernel that is wrong. A kernel that quietly cuts corners and stays inside the tolerance would walk straight through.

The winning kernel came from my hand sweep, and the agent-driven session worked a knob the server had already handed it. Machine verification is what I can claim here. Discovery is not.

Decode stays on the CPU. Full GPU offload runs into an upstream llama.cpp defect.

The board is dead, so everything above comes from archived logs, transcripts, and two notebooks that still re-run.

Conclusions

A language model proposed kernels for this GPU. A state machine decided what was true about them, and the model had no standing to argue.

That bought **883 simulation steps per second** beside a language model still decoding at **84%** of its solo rate, which beats every CPU-only arrangement I measured on both axes.

Faking a speedup needs a benchmark that will run on unverified code. In this graph there is no such transition, so a documented failure mode turned into an unreachable state.

Every number here came out of the gate ledger, and all of it reproduces from the raw logs at `github.com/msradam/seppa`.